LE RESPONSIVE DESIGN

TD Les bases de la programmation

Créez un dossier `07_responsive` contenant les deux fichiers `index.html` et `styles.css` fournis dans le dépôt Git, ainsi qu'un sous-dossier `images`.

A/ LE RESPONSIVE DESIGN ET LA BALISE DE VIEWPORT

A.1 Théorie

Le **responsive design** (ou « design adaptatif ») est une technique de conception d'interfaces qui fait en sorte que les éléments **s'adaptent à la taille de l'écran** : un même site, un même code HTML, mais un affichage différent sur mobile, tablette et ordinateur.

Concrètement, cela permet de :

- De positionner les blocs **de haut en bas** plutôt que côte à côte lorsque la largeur est réduite
- De **faire disparaître** sur mobile des blocs jugés moins importants
- Au contraire, de **faire apparaître** sur mobile de nouveaux éléments de navigation, comme le « menu burger »

Sans indication, un navigateur mobile affiche la page comme si l'écran mesurait 980px de large puis la réduit pour qu'elle tienne dans l'écran : **tout devient minuscule**. La balise ci-dessous, placée dans le `<head>`, corrige ce comportement :

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- `width=device-width` : la largeur de la page est **la largeur réelle de l'écran** de l'appareil ;
- `initial-scale=1.0` : la page s'affiche **sans zoom** au chargement.

Sans cette balise, aucune media query ne fonctionnera correctement sur un vrai mobile.

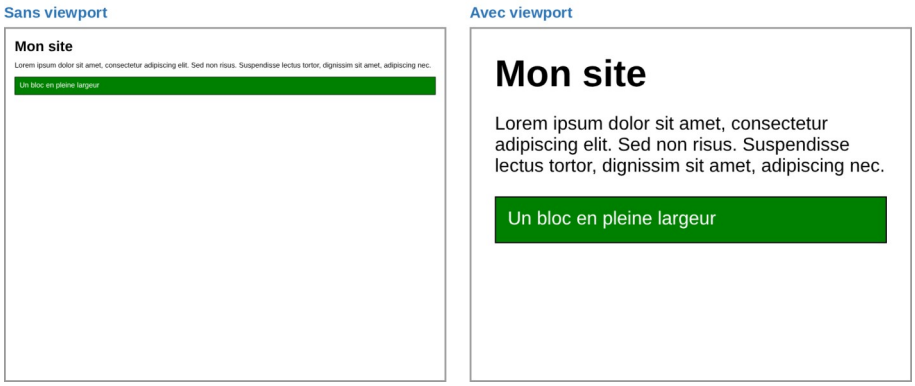


Figure 1: La même page sur un smartphone de 375px

A.2 Simuler différentes tailles d'écran

Pas besoin d'un téléphone pour tester : les outils de développement des navigateurs savent simuler n'importe quel écran.

- Ouvrez les outils de développement : **clic droit sur la page > Inspecter**, ou touche **F12**.
- Cliquez sur l'icône **« Responsive »** (un téléphone et une tablette) : à gauche dans Chrome, à droite dans Firefox. Raccourci : `Ctrl + Maj + M`.
- Choisissez un appareil dans la liste, ou saisissez une largeur précise en pixels.

A.3 Exercices

EXERCICE 1 Observer l'effet de la balise viewport

1. Ouvrez votre `index.html` dans le navigateur et activez le mode responsive.
2. Choisissez un **appareil prédéfini** (par exemple « iPhone SE »)
3. Retirez temporairement la balise `<meta name="viewport">`, enregistrez et rechargez la page
4. Comparez les deux affichages, puis remettez la balise en place. Pourquoi le texte devient-il minuscule sans la balise ? Quelle largeur le navigateur utilise-t-il alors pour afficher la page ?

B/ LES POINTS DE RUPTURE (MEDIA QUERIES)

B.1 Théorie

Un **point de rupture** est le moment où l'affichage d'un site change en fonction de la largeur (ou de la hauteur) de l'écran. En CSS, on le définit avec une **media query** : les règles écrites à l'intérieur ne s'appliquent **que si la condition est vraie**.

```
@media screen and (max-width: 480px) {

    div {
        display: block;
        width: 100%;
    }
}
```

- `screen` : la règle concerne les écrans (et non l'impression) ;
- `max-width: 480px` : s'applique **jusqu'à** 480 px de large (480 px compris) ;
- `min-width: 768px` : s'applique **à partir de** 768 px de large ;
- On peut combiner des conditions avec `and` : `(min-width: 768px) and (max-width: 991px)`.

Pourquoi après les règles normales ? Quand deux règles de même priorité se contredisent, c'est **la dernière écrite** qui s'applique. Une media query placée avant la règle normale serait donc écrasée par celle-ci.

B.2 Exemple

index.html

```
<div class="exercice">
    <h2>Partie B - Media query</h2>

    <div class="bloc three" id="exemple_media">
        Je suis bleu sur grand écran, rouge sur petit écran
    </div>
</div>
```

responsive.css

```
/* Partie B */
#exemple_media {
    font-size: 1.2em;
}

@media screen and (max-width: 600px) {
    #exemple_media {
        background-color: red;
        font-size: 1em;
    }
}
```

Observez : redimensionnez la fenêtre. le changement se produit **exactement** au passage des 600 px.



Figure 2: Résultat attendu : le bloc change de couleur et de taille de texte sous 600px

B.3 Exercices

EXERCICE 2 **Un texte, un écran**

Le bloc ci-dessous contient trois messages, tous masqués par défaut avec `display: none`. À l'aide de media queries, affichez **un seul** message (avec `display: block`) et changez la couleur de fond du bloc :

1. Moins de `768px` : message « mobile », fond `#f5c6aa` ;
2. De `768px` à `991px` : message « tablette », fond `#aaf5c6` ;
3. `992px` et plus : message « ordinateur », fond `#aad4f5`.

Question : *Que se passe-t-il exactement à 768 px et à 992 px ? Vos conditions se chevauchent-elles ?*

Exercice 2 – index.html

```
<div class="exercice">
    <h2>Exercice 2 - Quel écran ?</h2>

    <div class="bloc" id="ex2">
        <p class="msg-mobile">Vous êtes sur mobile</p>
        <p class="msg-tablette">Vous êtes sur tablette</p>
        <p class="msg-ordi">Vous êtes sur ordinateur</p>
    </div>
</div>
```

Exercice 2 – responsive.css

```
#ex2 p {
    margin: 0;
    font-weight: bold;
}

/* Par défaut, les trois messages sont masqués */
#ex2 .msg-mobile,
#ex2 .msg-tablette,
#ex2 .msg-ordi {
    display: none;
}

/* À compléter avec trois media queries */
```

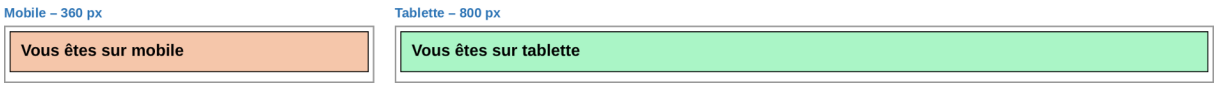


Figure 3: Résultats attendus de l'exercice 2 pour mobile et tablette



Figure 4: Résultat attendu de l'exercice 2 sur ordinateur

EXERCICE 3 Masquer un bloc sur mobile

Sur un petit écran, chaque pixel compte : on supprime ce qui n'est pas indispensable.

1. Ajoutez la section ci-dessous.
2. Masquez le bloc « Publicité » lorsque l'écran mesure **moins de 768px** de large.

Question : *Avec* `display: none`*, le bloc est-il simplement invisible, ou est-il complètement retiré de l'affichage ? Comparez avec* `visibility: hidden`*.*

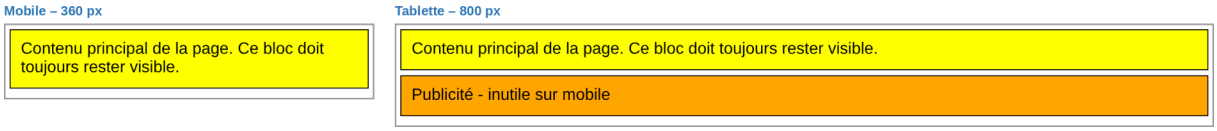


Figure 5: Résultats attendus de l'exercice 3

C/ LES TAILLES D'ÉCRAN ET LE « MOBILE FIRST »

C.1 Les points de rupture conventionnels

Il est impossible de viser chaque modèle d'appareil. On choisit donc **quelques points de rupture**, qui correspondent aux grandes familles d'écrans :

Type d'écran	Résolutions courantes	Point de rupture usuel
Mobile	360x640, 375x667, 360x720	576px
IPhone X, Pixel 2	375x812, 411x731	
Tablette	768x1024	768px
Ordinateur portable	1366x768	992px
Écran haute résolution	1920x1080	1200px

Conseil : plutôt que de viser des appareils, élargissez progressivement la fenêtre et placez un point de rupture **là où la mise en page « casse »** (lignes trop longues, blocs trop étroits…). Les valeurs ci-dessus sont de bons points de départ.

C.2 Théorie : le mobile first

La bonne pratique consiste à faire du **mobile first** : on écrit d'abord le CSS **pour le mobile** (sans media query), puis on ajoute des media queries en `min-width`, **dans l'ordre croissant**, pour enrichir l'affichage sur les écrans plus grands.

- L'affichage mobile est le plus simple (souvent une seule colonne) : il sert de base.
- On **ajoute** des règles pour les grands écrans, au lieu d'en **annuler** pour les petits : le code est plus court et plus lisible.

Aujourd'hui, la majorité des visites d'un site se font sur mobile.

```
/* Mobile first : la base, pour les mobiles */
.bloc {
    width: 100%;
}

/* Tablette : à partir de 768px */
@media screen and (min-width: 768px) {
    .bloc { ... }
}

/* Ordinateur : à partir de 992px */
@media screen and (min-width: 992px) {
    .bloc { ... }
}
```

C.3 Exercices

EXERCICE 4 **Trois blocs côte à côte**

Ajoutez la section ci-dessous, puis, en **mobile first** :

1. Sur mobile, les trois blocs occupent **toute la largeur** de la page, les uns sous les autres (`display: block`) ;
2. À partir de `992px` (« ordinateur »), chaque bloc occupe **un tiers** de la largeur et ils se placent côte à côte : utilisez `display: inline-block` et une largeur de `32%`.

Question : *Essayez avec une largeur de `33.33%` : que se passe-t-il ? Pourquoi ?*

Exercice 4 – index.html

```
<div class="exercice">
    <h2>Exercice 4 - Trois blocs</h2>

    <div id="ex4">
        <div class="bloc one">Bloc 1</div>
        <div class="bloc two">Bloc 2</div>
        <div class="bloc three">Bloc 3</div>
    </div>
</div>
```

Exercice 4 – responsive.css

```
/* Mobile first : à compléter */
#ex4 .bloc {

}

/* Point de rupture "ordinateur" : à compléter */
```

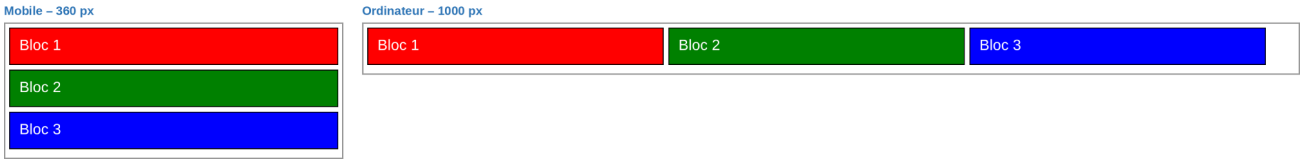


Figure 6: Résultats attendus de l'exercice 4

EXERCICE 5 **Ajouter un palier tablette**

Ajoutez une section de six blocs (id `ex5`), puis, toujours en mobile first :

1. Mobile : **1 bloc** par ligne ;
2. À partir de `768px` : **2 blocs** par ligne (largeur `48%`) ;
3. À partir de `992px` : **3 blocs** par ligne (largeur `32%`).

Question : *Dans quel ordre devez-vous écrire les deux media queries ? Inversez-les pour vérifier votre réponse.*

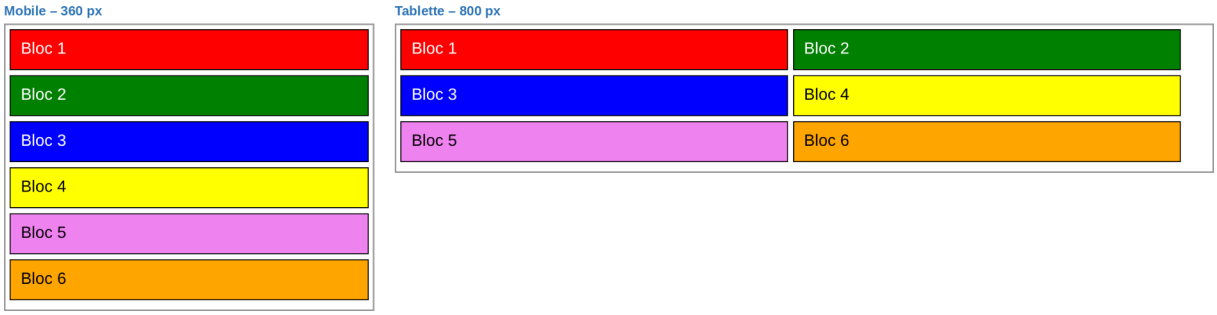


Figure 7: Résultats attendus de l'exercice 5 sur mobile et tablette



Figure 8: Résultat attendu de l'exercice 5 sur ordinateur

D/ LE MODE D'AFFICHAGE

D.1 Théorie

La propriété `display` définit le **mode d'affichage** d'un élément, et la disposition de ses enfants. C'est l'outil de base du responsive : en changer dans une media query transforme complètement la mise en page.

- `block` : l'élément occupe **toute la largeur** disponible et passe à la ligne (`div`, `p`, `h1`, `li`…) ;
- `inline` : l'élément reste **dans la ligne de texte** ; on ne peut pas lui donner de largeur ni de hauteur (`span`, `a`, `strong`…) ;
- `inline-block` : l'élément reste dans la ligne, **mais accepte** une largeur, une hauteur et des marges ;
- `none` : l'élément est **retiré de l'affichage** ;
- `flex` et `grid` : des modes de mise en page avancés

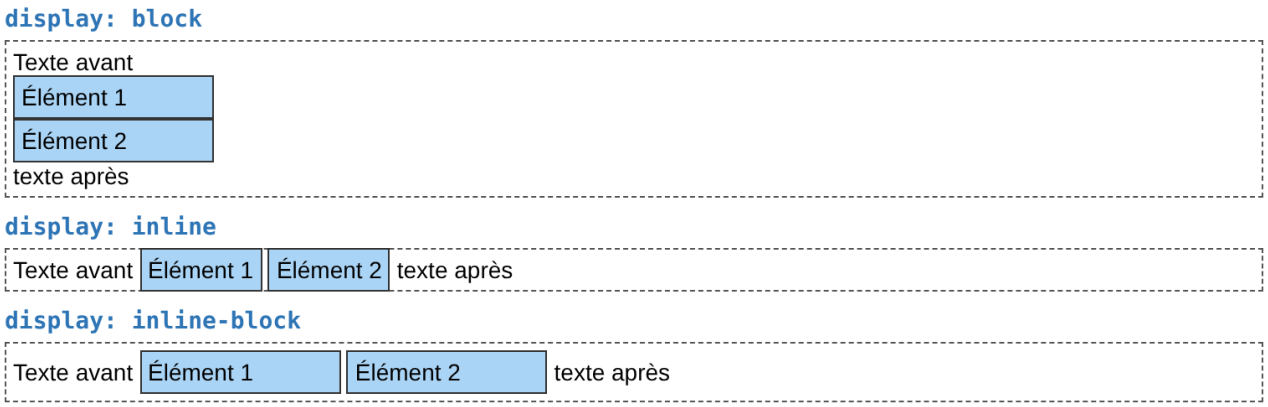


Figure 9: Trois éléments identiques, de largeur 120 px, avec trois modes d'affichage différents

Observez : en `inline`, la largeur de 120 px est **ignorée** ; en `block`, chaque élément est seul sur sa ligne.

D.2 Exercices

EXERCICE 6 **Un menu vertical ou horizontal**

Ajoutez la section et le CSS ci-dessous, puis :

1. Sur mobile, les entrées du menu sont **les unes sous les autres**, sur toute la largeur ;
2. À partir de `768px`, elles sont **côte à côte**, sans bordure basse.

Question : *Quelle valeur de `display` les éléments `<li>` ont-ils par défaut ?*

Exercice 6 – index.html

```
<div class="exercice">
    <h2>Exercice 6 - Menu adaptatif</h2>

    <ul class="menu" id="ex6">
        <li>Accueil</li>
        <li>Articles</li>
        <li>Galerie</li>
        <li>Contact</li>
    </ul>
</div>
```

Exercice 6 – responsive.css

```
.menu {
    list-style: none;
    margin: 0;
    padding: 0;
    background-color: #2c3e50;
}

.menu li {
    color: white;
    padding: 10px 15px;
    border-bottom: 1px solid #34495e;
}

/* À compléter */
```



Figure 10: Résultat attendu de l'exercice 6

EXERCICE 7 Faire apparaître un menu burger

Sur mobile, on remplace souvent le menu par un simple bouton « ☰ Menu » (le « menu burger ») :

1. sur mobile : le bouton est **visible** et le menu est **masqué** ;
2. à partir de `768px` : le bouton est **masqué**, et le menu est **visible**, avec ses entrées côte à côte.

Le code `☰` affiche le symbole ☰. Pour ouvrir réellement le menu au clic, il faut du JavaScript… ou l'astuce présentée dans la partie « Pour aller plus loin ».

Exercice 7 – index.html

```
<div class="exercice">
    <h2>Exercice 7 - Bouton menu burger</h2>

    <div id="ex7">
        <div class="burger">&#9776; Menu</div>
        <ul class="menu">
            <li>Accueil</li>
            <li>Articles</li>
            <li>Galerie</li>
            <li>Contact</li>
        </ul>
    </div>
</div>
```

Exercice 7 – responsive.css

```
#ex7 .burger {
    background-color: #2c3e50;
    color: white;
    padding: 10px 15px;
    font-weight: bold;
}

/* À compléter (le style .menu de l'exercice 6 s'applique aussi ici) */
```

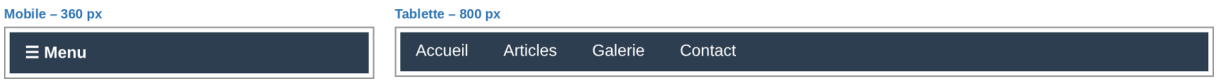


Figure 11: Résultat attendu de l'exercice 7

E/ LES UNITÉS RELATIVES

E.1 Théorie

Une taille en **pixels** (`px`) est **fixe** : elle ne s'adapte ni à l'écran, ni aux réglages de l'utilisateur. Les unités **relatives** dépendent d'une autre valeur, et s'adaptent donc d'elles-mêmes :

Unité	Relative à…	Exemple
`%`	La taille de l'élément parent	`width: 50%`
`em`	La taille de police de l'élément (ou de son parent)	`padding: 1em`
`rem`	La taille de police de la racine `<html>` (16px par défaut)	`font-size: 1.5rem`
`vw / vh`	1% de la largeur / de la hauteur de la fenêtre	`width: 50vw;` `min-height: 100vh;`

- **`rem` pour les tailles de texte et les espacements** : changer la taille de police de `<html>` agrandit alors toute la page d'un coup, et la page respecte la taille de police choisie par l'utilisateur dans son navigateur ;
- **`em`** est utile pour un espacement qui doit suivre la taille du texte de l'élément (le `padding` d'un bouton), mais attention : les `em` **se multiplient** quand on les imbrique ;
- **`clamp(min, idéal, max)`** borne une valeur fluide : `font-size: clamp(1.5rem, 5vw, 3rem);` suit la largeur de la fenêtre (`5vw`), sans descendre sous `1.5rem` ni dépasser `3rem`.

E.2 Exercices

EXERCICE 8 **em ou rem ?**

Ajoutez la section ci-dessous : deux séries de blocs imbriqués les uns dans les autres.

1. Dans la colonne de gauche, donnez à chaque `.niveau` une taille de police de `1.2em`.
2. Dans la colonne de droite, donnez-leur une taille de `1.2rem`.
3. Calculez la taille du texte « Niveau 4 » dans chaque colonne, puis vérifiez avec les outils de développement (onglet « Calculé »).

Question : *Pourquoi le texte grossit-il à chaque niveau avec `em`, et pas avec `rem` ?*

Exercice 8 – index.html

```
<div class="exercice">
    <h2>Exercice 8 - em ou rem ?</h2>

    <div class="colonne" id="ex8_em">
        <p class="label">Avec em</p>
        <div class="niveau">Niveau 1
            <div class="niveau">Niveau 2
                <div class="niveau">Niveau 3
                    <div class="niveau">Niveau 4</div>
                </div>
            </div>
        </div>
    </div>
    <div class="colonne" id="ex8_rem">
        <p class="label">Avec rem</p>
        <div class="niveau">Niveau 1
            <div class="niveau">Niveau 2
                <div class="niveau">Niveau 3
                    <div class="niveau">Niveau 4</div>
                </div>
            </div>
        </div>
    </div>
</div>
```

Exercice 8 – responsive.css

```
.colonne {
    display: inline-block;
    width: 48%;
    vertical-align: top;
}

.niveau {
    border-left: 3px solid #2e75b6;
    padding-left: 10px;
    margin-top: 5px;
}

#ex8_em .niveau {
```

```
    /* À compléter : 1.2 fois la taille du parent */
}

#ex8_rem .niveau {
    /* À compléter : 1.2 fois la taille de la racine */
}
```

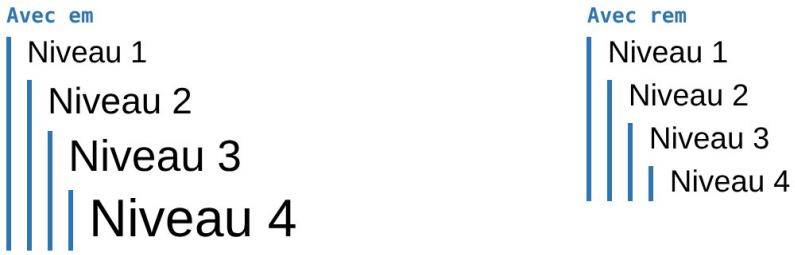


Figure 12: Résultat attendu de l'exercice 8

EXERCICE 9 **Tout agrandir d'un coup**

Ajoutez la section et le CSS ci-dessous : toutes les tailles sont en `rem`.

1. À partir de `992px`, passez la taille de police de la racine (`html`) à `125%` (soit 20 px au lieu de 16 px) ;
2. Observez l'article, mais aussi le reste de votre page ;
3. Dans les réglages de votre navigateur, augmentez la taille de police par défaut, et observez à nouveau.

Exercice 9 – index.html

```
<div class="exercice">
    <h2>Exercice 9 - Tout agrandir d'un coup</h2>

    <div class="article-rem" id="ex9">
        <h3>Titre de l'article</h3>
        <p>Toutes les tailles de cet article sont exprimées en rem :
           elles dépendent toutes de la taille de police de la racine.</p>
        <div class="bouton-rem">Lire la suite</div>
    </div>
</div>
```

Exercice 9 – responsive.css

```
.article-rem {
    border: 1px solid #999;
    padding: 1rem;
}

.article-rem h3 {
    font-size: 1.5rem;
    margin: 0 0 0.5rem;
}

.article-rem p {
```

```
    font-size: 1rem;
    margin: 0 0 1rem;
}

.bouton-rem {
    display: inline-block;
    font-size: 1rem;
    padding: 0.5rem 1rem;
    background-color: #2e75b6;
    color: white;
}

/* À compléter */
```



EXERCICE 10 Un titre fluide avec clamp()

Ajoutez la section ci-dessous, puis :

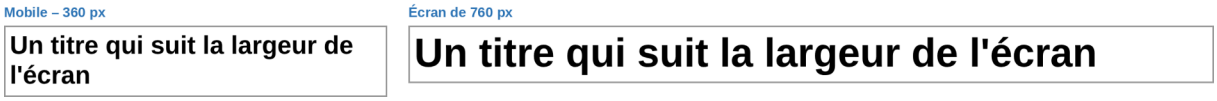
1. donnez au titre une taille de police qui vaut `5vw`, mais jamais moins de `1.5rem`, ni plus de `3rem` ;
2. redimensionnez lentement la fenêtre et observez le titre.

Question : *À partir de quelle largeur de fenêtre le titre atteint-il sa taille maximale (48 px) ? Et en dessous de quelle largeur reste-t-il à sa taille minimale (24 px) ?*

Exercice 10 – index.html

```
<div class="exercice">
    <h2>Exercice 10 - Un titre fluide</h2>

    <div id="ex10">
        <h3 class="titre-fluide">Un titre qui suit la largeur de l'écran</h3>
    </div>
</div>
```



F/ DES ÉLÉMENTS ET DES IMAGES FLUIDES

Entre deux points de rupture, les éléments doivent aussi **s'adapter en continu**.

- Éviter les **largeurs fixes** en pixels pour les grands blocs : préférer les **pourcentages** ;
- Utiliser `max-width` pour limiter la largeur sur grand écran sans bloquer le rétrécissement : `width: 100%; max-width: 800px;` ;
- Une image garde par défaut sa **taille réelle** : une photo de 1600 px de large dépasse de l'écran d'un mobile. `max-width: 100%; height: auto;` l'empêche de dépasser de son conteneur, en conservant ses proportions ;
- Quand une image doit remplir un cadre de taille fixe (vignette, bannière), `object-fit` choisit comment elle s'y adapte : `fill` (par défaut, l'image est **déformée**), `contain` (elle est **entière**, avec des bandes vides), `cover` (elle **remplit** le cadre, en étant recadrée).

```
img {
    max-width: 100%; /* jamais plus large que son conteneur */
    height: auto; /* conserve les proportions */
}
```

Une image adaptée à chaque écran. Pourquoi charger une photo de 1600 px sur un téléphone ? HTML propose deux mécanismes :

- `srcset` et `sizes` : on fournit **plusieurs tailles** du même fichier, et le navigateur télécharge la plus adaptée à l'écran (et à sa densité de pixels) ;
- `<picture>` : on choisit **une image différente** selon une media query, par exemple un cadrage portrait sur mobile et paysage sur ordinateur. La balise `<img>` finale est obligatoire : c'est elle qui est affichée et mise en forme.

```
<!-- srcset : le navigateur choisit le fichier (400w = 400 px de large) -->
<img src="images/photo-800.jpg"
     srcset="images/photo-400.jpg 400w,
             images/photo-800.jpg 800w,
             images/photo-1600.jpg 1600w"
     sizes="(min-width: 992px) 50vw, 100vw"
     alt="Paysage de montagne">

<!-- picture : on choisit l'image selon l'écran -->
<picture>
    <source media="(min-width: 768px)" srcset="images/photo-large.jpg">
    <img src="images/photo-mobile.jpg" alt="Paysage de montagne">
</picture>
```

N'oubliez jamais l'attribut `alt` : il décrit l'image pour les personnes qui ne la voient pas (lecteurs d'écran), ou quand elle ne se charge pas.

G/ LES AUTRES CARACTÉRISTIQUES

G.1 Théorie

Une media query peut tester bien d'autres caractéristiques que la largeur de l'écran :

- `(orientation: portrait)` ou `(orientation: landscape)` : l'écran est-il plus haut que large, ou l'inverse (téléphone tourné) ?
- `(prefers-color-scheme: dark)` : l'utilisateur a-t-il choisi un **thème sombre** dans son système ?
- `(hover: hover)` : l'appareil possède-t-il un pointeur capable de **survoler** (une souris) ? Sur un écran tactile, il n'y a pas de survol.
- `print` à la place de `screen` : des règles qui ne s'appliquent **qu'à l'impression** (masquer le menu, le fond…).

```
@media (orientation: landscape) {
    .galerie img { width: 48%; }
}

@media (prefers-color-scheme: dark) {
    body {
        background-color: #121212;
        color: #eeeeee;
    }
}
```

Pour tester sans changer de système : dans Chrome, ouvrez le menu ⋮ des outils de développement > **Plus d'outils > Rendu**, puis « Émuler la fonctionnalité CSS prefers-color-scheme ». Dans Firefox, utilisez les icônes soleil et lune de l'onglet **Inspecteur**. Pour l'orientation, le mode responsive possède un bouton de **rotation**.

G.2 Exercices

EXERCICE 11 **Portrait ou paysage ?**

Ajoutez la section ci-dessous, puis, **sans utiliser la largeur de l'écran** :

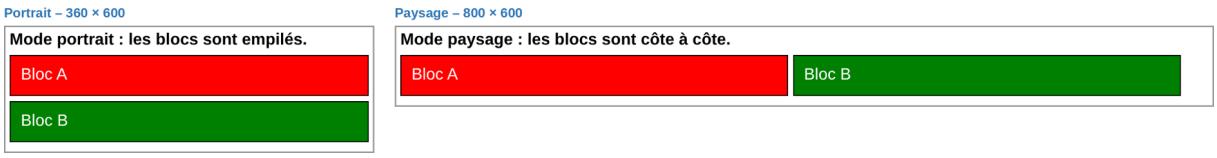
1. En portrait : seul le premier message est affiché, et les blocs sont les uns sous les autres ;
2. En paysage : seul le second message est affiché, et les deux blocs sont côte à côte (`inline-block`, `48%`).

Testez avec le bouton de rotation du mode responsive. Une fenêtre d'ordinateur est-elle en portrait ou en paysage ?

Exercice 11 – index.html

```
<div class="exercice">
    <h2>Exercice 15 - Portrait ou paysage ?</h2>

    <div id="ex15">
        <p class="msg-portrait">Mode portrait : les blocs sont empilés.</p>
        <p class="msg-paysage">Mode paysage : les blocs sont côte à côte.</p>
        <div class="bloc one">Bloc A</div>
        <div class="bloc two">Bloc B</div>
    </div>
</div>
```



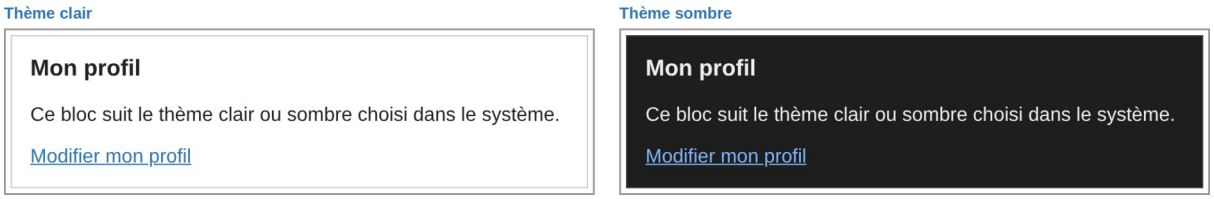
EXERCICE 12 Un thème sombre automatique

Ajoutez la section et le CSS ci-dessous, puis, lorsque l'utilisateur a choisi un **thème sombre** :

1. le fond de la carte devient `#1e1e1e`, le texte `#eeeeee` et la bordure `#444444` ;
2. le lien devient `#7fb8ff`, pour rester lisible sur fond sombre.

Testez avec l'émulation de `prefers-color-scheme` des outils de développement.

Question : *Pourquoi faut-il aussi changer la couleur du lien, et pas seulement celle du fond et du texte ?*



Exercice 12 – index.html

```
<div class="exercice">
    <h2>Exercice 16 - Un thème sombre automatique</h2>

    <div class="carte-theme" id="ex16">
        <h3>Mon profil</h3>
        <p>Ce bloc suit le thème clair ou sombre choisi dans le système.</p>
        <a href="#">Modifier mon profil</a>
    </div>
</div>
```

Exercice 12 – responsive.css

```
.carte-theme {
    background-color: #ffffff;
    color: #222222;
    border: 1px solid #cccccc;
    padding: 15px;
}

.carte-theme h3 { margin-top: 0; }
.carte-theme a { color: #2e75b6; } /* puis à compléter */
```

H/ L'ERGONOMIE SUR MOBILE

H.1 Théorie

Un site responsive ne se contente pas de « tenir » dans l'écran : il doit être **agréable à utiliser au doigt**, sur un petit écran, parfois en plein soleil ou d'une seule main.

- **Des zones tactiles assez grandes :** un doigt est bien moins précis qu'une souris. Les boutons et liens doivent mesurer au moins **44 × 44 px** (recommandation d'Apple et du niveau AAA des règles d'accessibilité WCAG ; le niveau AA exige au minimum 24 × 24 px), et être **espacés** ;
- **Des textes lisibles :** au moins `16px` (`1rem`) pour le texte courant, avec un interlignage confortable (`line-height: 1.5`) ;
- **Des champs de formulaire en 16 px minimum :** en dessous, Safari sur iPhone **zoome automatiquement** sur le champ, et l'utilisateur doit dézoomer ;
- **Le bon type de champ :** `type="email"`, `type="tel"` ou `type="number"` affichent un clavier adapté sur mobile ;
- **Rien d'indispensable au survol :** sur un écran tactile, un menu qui ne s'ouvre qu'au survol est inutilisable ;
- **Ne pas bloquer le zoom :** la page doit rester utilisable quand l'utilisateur zoome jusqu'à 200 %.

Checklist de test à appliquer à chaque page :

- Tester aux largeurs `320px`, `375px`, `768px`, `1024px` et `1440px`, puis en mode paysage ;
- Vérifier qu'il n'y a **aucun défilement horizontal** ;
- Vérifier que les textes sont lisibles **sans zoomer**, et que les images ne sont **ni déformées, ni débordantes** ;
- Vérifier que chaque bouton et chaque lien se touchent **facilement au doigt** ;
- Zoomer à 200 % (`Ctrl` + `+`) : tout le contenu doit rester accessible ;
- Si possible, tester sur un **vrai téléphone** : l'émulation ne remplace pas le test réel.

H.2 Exercice

EXERCICE 13 **Un formulaire utilisable au doigt**

Le formulaire ci-dessous « tient » dans un écran de mobile, mais il est pénible à utiliser. Testez-le à `375px`, puis corrigez-le :

1. Les champs occupent **toute la largeur** et leur texte mesure **au moins 16 px** ;
2. Le bouton occupe toute la largeur et mesure **au moins 44 px** de haut ;
3. Les deux liens ont une zone cliquable d'**au moins 44 px** de haut, et sont bien séparés ;
4. Le champ e-mail affiche le **clavier adapté** sur mobile (modifiez le HTML).



Exercice 13 – index.html

```
<div class="exercice">
    <h2>Exercice 17 - Un formulaire utilisable au doigt</h2>

    <form class="connexion" id="ex17">
        <label for="email">E-mail</label>
        <input type="text" id="email">
        <label for="mdp">Mot de passe</label>
        <input type="password" id="mdp">
        <button type="submit">Se connecter</button>
        <a href="#">Mot de passe oublié ?</a>
        <a href="#">Créer un compte</a>
    </form>
</div>
```

Exercice 13 – responsive.css

```
/* CSS de départ : à corriger */
#ex17 {
    font-size: 12px;
}

#ex17 label,
#ex17 input,
#ex17 button {
```

```
    display: block;
    margin-bottom: 4px;
}

#ex17 input {
    width: 150px;
    font-size: 12px;
    padding: 2px;
}

#ex17 button {
    font-size: 11px;
    padding: 2px 4px;
}

#ex17 a {
    font-size: 11px;
    margin-right: 2px;
}
```